

SQL Assignment — Group 9: Playlists Table and Stored Procedure

Group 9 — Playlists Table & Stored Procedure

1. CREATE TABLE playlists

Create a MySQL table called *playlists* with columns: *playlist_id* (primary key), *user_id* (foreign key), *name* (varchar), and *created_at* (datetime).

```
CREATE TABLE playlists (  
  playlist_id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id     INT,  
  name        VARCHAR(100),  
  created_at  DATETIME,  
  FOREIGN KEY (user_id) REFERENCES SpotifyUser(user_id)  
);
```

Explanation: DATETIME (unlike DATE used in Group 1) stores both the date and time a playlist was created. The foreign key ties each playlist back to the user who owns it, referencing the `SpotifyUser` table created earlier.

2. Insert 3 sample playlists

Insert three sample records: 'Workout Mix', 'Chill Vibes', and 'Top Hits', with realistic data.

```
INSERT INTO playlists (user_id, name, created_at) VALUES  
  (1, 'Workout Mix', '2026-08-01 07:30:00'),  
  (1, 'Chill Vibes', '2026-08-05 21:15:00'),  
  (2, 'Top Hits',   '2026-08-10 12:00:00');
```

Explanation: The DATETIME values include both a date and a time (in 'YYYY-MM-DD HH:MM:SS' format), simulating realistic playlist-creation timestamps, e.g. an early-morning workout playlist versus a late-evening chill playlist.

3. UPDATE to rename a playlist for a specific user

Write an UPDATE statement to rename the 'Chill Vibes' playlist to 'Evening Chill' for a specific *user_id*.

```
UPDATE playlists  
SET name = 'Evening Chill'  
WHERE name = 'Chill Vibes' AND user_id = 1;
```

Explanation: Combining `name = 'Chill Vibes'` with `user_id = 1` in the WHERE clause ensures only that specific user's playlist is renamed — important if multiple users could have a playlist with the same name.

4. DELETE a playlist for a given user

Write a DELETE statement to remove a playlist named 'Workout Mix' for a given *user_id*.

```
DELETE FROM playlists
```

```
WHERE name = 'Workout Mix' AND user_id = 1;
```

Explanation: As with the update above, both conditions together in the `WHERE` clause make sure the `DELETE` targets only the intended row — this specific user's "Workout Mix" playlist — rather than every playlist with that name across all users.

5. Stored Procedure — GetMonthlyPlaylistCount

Create a stored procedure named *GetMonthlyPlaylistCount* that takes a *user_id* and a month (integer) as input, and returns the total number of playlists the user created in that month.

```
DELIMITER //
```

```
CREATE PROCEDURE GetMonthlyPlaylistCount(  
    IN p_user_id INT,  
    IN p_month    INT  
)  
BEGIN  
    SELECT COUNT(*) AS playlist_count  
    FROM playlists  
    WHERE user_id = p_user_id  
        AND MONTH(created_at) = p_month;  
END//
```

```
DELIMITER ;
```

```
-- Example call: how many playlists did user 1 create in August (month 8)?  
CALL GetMonthlyPlaylistCount(1, 8);
```

Explanation: The procedure accepts two input parameters, `p_user_id` and `p_month`, prefixed with `p_` to distinguish them from column names. Inside, `MONTH(created_at)` extracts just the month number (1-12) from each playlist's `created_at` timestamp, and the query counts how many of that user's playlists fall in the requested month. Calling `CALL GetMonthlyPlaylistCount(1, 8);` reuses this logic for any user/month combination without rewriting the query each time.